Performance Optimization of UAV-assisted NTN Communication using Open-Loop and Closed-Loop Machine Learning with NetSim Integration

Vinay Kusumanchi (IMT2023608)

Abstract

This document presents the complete workflow of implementing open-loop and closed-loop machine learning (ML) for performance optimization in a Non-Terrestrial Network (NTN) scenario using a UAV base station. The work spans NTN concepts, scenario design in NetSim, dataset creation, ML modelling, Python-NetSim interfacing, and iterative closed-loop performance evaluation for both one-user and five-user cases. Results demonstrate clear improvements in throughput, delay, and jitter for both scenarios, highlighting the effectiveness of closed-loop optimization. Although the UAV link provides strong LoS conditions, the 5-user case exhibits slightly higher delay and jitter due to resource sharing, confirming realistic load-dependent behaviour.

1 Introduction

Non-Terrestrial Networks (NTNs) are communication systems that use platforms located above the Earth's surface including satellites, High Altitude Platforms (HAPS) and Unmanned Aerial Vehicles (UAVs). NTNs are becoming a core component of the 6G vision as standardized from 3GPP Release 17.

Motivation for NTNs

- Provide connectivity in rural or remote regions.
- Enable maritime and aviation communication.
- Support emergency/disaster recovery networks.
- Enable defense and tactical communication.
- Provide massive IoT connectivity in uncovered areas.

2 What are Non-Terrestrial Networks (NTNs)?

Non-Terrestrial Networks (NTNs) are communication systems in which at least one network node such as a base station, relay node, or terminal is located above the Earth's

surface. Unlike traditional terrestrial networks composed of fixed ground-based stations, NTNs operate using aerial or spaceborne platforms such as satellites, High Altitude Platforms (HAPS), and Unmanned Aerial Vehicles (UAVs). NTNs have become a key enabler of the 6G vision, as highlighted in 3GPP Release 17, due to their ability to provide global coverage, high mobility support, and resilient communication during emergencies.

NTNs integrate seamlessly with terrestrial networks and support a wide range of applications, from broadband connectivity and IoT deployments to remote sensing and disaster response. Their altitude dependent characteristics allow significant coverage footprints, making them especially suitable for remote, rural, and underserved regions where ground infrastructure is limited.

3 Why NTNs?

The need for NTNs arises from several practical, economic, and strategic motivations:

3.1 Global and Remote-Area Coverage

Large parts of the Earth lack terrestrial cellular connectivity due to economic or geographic barriers. NTNs can provide service to:

- Rural villages
- Maritime users
- Remote islands
- Desert or mountain regions

Satellites and HAPS offer ultra-wide coverage that terrestrial towers cannot achieve.

3.2 Mobility Support for Aviation and Maritime

Commercial airlines, cargo ships, fishing vessels, and naval fleets require continuous broadband connectivity. NTNs, especially LEO satellites enable high speed communication across oceans and international air routes.

3.3 Emergency and Disaster Recovery

During natural disasters such as earthquakes, floods, or cyclones, terrestrial infrastructure may become damaged. UAVs and HAPS can quickly restore network coverage, enabling:

- Search and rescue operations
- Disaster management connectivity
- Emergency communication services

3.4 Defense and Tactical Communication

MILCOM applications rely heavily on secure, jam resistant, and high mobility links. NTNs ensure resilient communication for:

- Border surveillance
- Drone operations
- Tactical edge networks

3.5 IoT and Machine-Type Communication

NTNs support massive IoT deployments in regions where terrestrial gateways are not viable. LEO constellations enable low latency IoT communication for:

- Environmental sensors
- Agricultural monitoring
- Logistics tracking
- Pipeline and grid monitoring

4 NTN Platforms

NTN systems use several types of aerial and spaceborne platforms. Their coverage, latency, and mobility characteristics vary significantly.

4.1 Geostationary Orbit (GEO)

- Altitude: 35,786 km
- Coverage: One GEO satellite covers one-third of the Earth
- Latency: Very high (500–600 ms)
- Example: INSAT, GSAT

GEO satellites provide stable coverage but suffer from long propagation delay, making them unsuitable for interactive applications.

4.2 Medium Earth Orbit (MEO)

- Altitude: 2,000–20,000 km
- Latency: Moderate (100–150 ms)
- Use cases: Navigation (GPS, Galileo), broadband

4.3 Low Earth Orbit (LEO)

- Altitude: 500–2,000 km
- Latency: Low (20–40 ms)
- Highly mobile satellites requiring tracking
- Examples: Starlink, OneWeb

4.4 High Altitude Platforms (HAPS)

- Altitude: Around 20 km (stratosphere)
- Solar powered, long endurance aircraft or balloons
- Lower latency than satellites

4.5 Unmanned Aerial Vehicles (UAVs)

UAVs serve as low altitude communication nodes with:

- Flexible deployment
- Rapid mobility
- On-demand coverage

In this project, the UAV acts as an aerial base station communicating with ground users.

5 NTN Requirements

NTNs must satisfy several technical requirements to ensure reliable operation:

5.1 Latency Constraints

Different platforms introduce varying propagation delays. LEO satellites and UAVs offer low latency compared to GEO systems.

5.2 Throughput and Capacity

High throughput NTN links must support:

- Broadband services
- Backhaul for remote towers
- IoT and M2M communication

5.3 Mobility and Handover Management

LEO satellites and UAVs move rapidly. Efficient handover and trajectory prediction mechanisms are required for seamless connectivity.

5.4 Interference Management

NTNs share spectrum with terrestrial networks, making:

- Beamforming,
- Power control, and
- Frequency planning

essential for interference avoidance.

5.5 Energy Efficiency

Power is limited on satellites and UAVs. Efficient routing, duty cycling, and adaptive communication protocols are necessary.

5.6 Synchronization and Doppler Compensation

High mobility causes severe Doppler shifts, especially in LEO constellations, requiring specialized synchronization techniques.

6 Key Research Areas in NTNs

NTNs involve a wide range of research topics. Major areas include:

6.1 Routing Protocols

NTN routing must handle:

- Dynamic topology
- Variable link quality
- Rapid mobility of nodes

6.2 Resource Allocation

Efficient allocation of power, spectrum, and beamforming resources is critical to maximize system throughput and fairness.

6.3 Random Access and Scheduling

NTN users experience variable link conditions and congestion. Adaptive random access techniques are required.

6.4 AI/ML enabled NTN Optimization

Machine learning techniques improve:

- Trajectory planning
- Routing decisions
- Power control
- Traffic prediction

6.5 Handover Management

Especially significant in LEO constellations where satellites move rapidly relative to users.

6.6 Security and Authentication

NTNs are vulnerable to jamming and spoofing; secure communication is essential.

7 Literature Survey

Table 1 summarizes key literature findings relevant to NTN routing, machine learning adoption, and aerial communication systems.

Table 1: Summary of Literature Survey

Paper	**Key Contribution**
Zhao et al. (2022)	Link prediction–based routing method for LEO satellite networks to improve path stability.
Hosseini & Jamalipour (2022)	Comprehensive survey of constellation routing methods in LEO networks and their limitations.
Wan et al. (2023)	Proposed distributed on-demand routing suitable for dynamic LEO topologies.
3GPP Release–17 NTN Study	Standardization of NTN architecture, frequency bands, and performance requirements for 6G.

Machine Learning–based NTN Literature Summary

Table 2: Summary of ML Models and Their Objectives in NTN-related Research

Paper / Work	**ML Model**	**Objective**	**Dataset**	**Key Outcome**
Random Forest Classification (OpenML SatImage)	Random Forest	Classify satellite surface types and learn feature importance	OpenML "satimage" dataset	Achieved 91.6% accuracy; shows RF can model nonlinear relationships useful for NTN prediction
Q-Learning (FrozenLake Simulation)	Q-Learning (RL)	Learn optimal policy via rewards for routing-like decisions	FrozenLake environment	Q-values converge to optimal path; demonstrates RL suitability for dynamic NTN routing
Q-Routing Simulation	Q-Routing (Model-free RL)	Enable nodes to learn best next-hop using estimated delay	Custom simulated multi-node network	Nodes learn optimal neighbors; adapts quickly to topology changes similar to LEO satellites
AI-driven NTN Optimization (Survey)	RF / RL / Deep RL	Traffic prediction, routing, handover optimization	Multiple public datasets	ML improves prediction accuracy and reduces delay under dynamic NTN conditions

8 Machine Learning Concepts for NTNs

Machine learning (ML) plays a transformative role in optimizing NTN systems. ML methods can learn from network data, predict future conditions, and adapt system parameters in real time.

8.1 Supervised Learning

Supervised learning models learn from labeled data. In NTN optimization, supervised models predict:

- Throughput
- Delay
- Jitter
- Packet loss

Random Forest Regression is one such technique.

8.2 Random Forest: Theoretical Background

Random Forest is an ensemble learning technique consisting of multiple decision trees built on randomly selected subsets of the data. Key features include:

- Bagging (bootstrap aggregation)
- Random feature selection
- Averaging predictions to reduce overfitting

This algorithm is robust for network performance prediction tasks due to its ability to model nonlinear relationships.

8.3 Reinforcement Learning (RL)

RL is useful when the network must learn optimal actions through trial and error. RL operates on:

- States
- Actions
- Rewards

The agent learns to maximize cumulative reward.

8.4 Q-Learning

Q-learning is a model free RL method. It maintains a Q-table that stores the value of taking an action in a particular state. Over time, the model converges to optimal action values.

8.5 Q-Routing

Q-routing adapts Q-learning concepts to routing. Nodes maintain Q-values for each neighbor representing estimated delivery time. As packets flow through the network:

- Nodes update Q-values
- Routing decisions improve
- Congestion is avoided

Although not implemented in this project, these techniques influence advanced NTN routing research.

9 Routing Methods in NTNs

- **Centralized Routing:** High accuracy, higher overhead.
- **Distributed Routing:** Low latency, scalable.
- **Hybrid Routing:** Combines SDN global view with local decisions.

10 Machine Learning for NTN Optimization

10.1 Random Forest Regression

Used to model performance (throughput, delay, jitter) as a function of:

- Packet Size (Bytes)
- Inter-Arrival Time (IAT, in μs)

10.2 Closed-Loop Concept

Instead of predicting once (open-loop), closed-loop ML:

- retrains after each NetSim run,
- updates parameters,
- and gradually converges towards an optimal pair.

11 NetSim Scenario Setup

11.1 Topology

- One UAV Base Station.
- One or five ground users depending on scenario.
- UAV mobility defined using a trace file.
- CBR traffic flows between UAV and users.

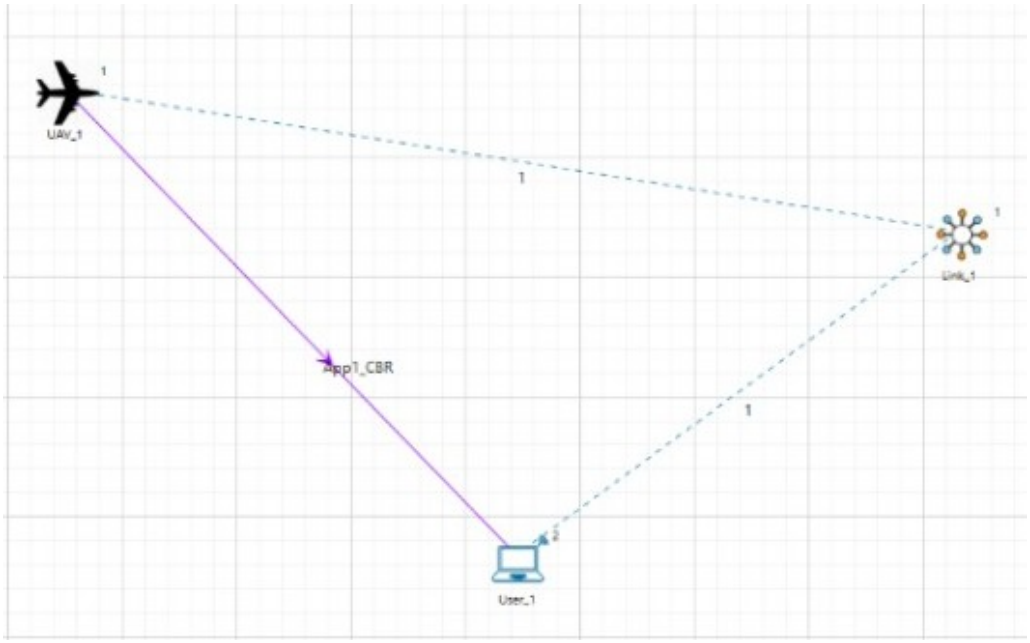

Figure 1: NetSim Topology Showing UAV and one User

11.2 Traffic Configuration

Each user receives a CBR flow configured with:

- UDP transport protocol
- Adjustable Packet Size
- Adjustable Inter-arrival Time

NetSim - Rapid Application Configurator

New Application | Delete Selected | Export list to Excel | Import list from Excel | Close

CBR 5 | CUSTOM 0 | VOICE 0 | VIDEO 0 | FTP 0 | HTTP 0 | EMAIL 0 | DATABASE 0 | GAMING 0

Application ID	Application Name	Source ID	Destination ID	Start Time (s)	End Time (s)	Transport protocol	Packet Size (B)	Interarrival Time (µs)	Generation rate
1	App1_CBR	1	2	5	100000	UDP	1024	5000	1.64 Mbps
2	App2_CBR	1	3	5	100000	UDP	1460	3000	3.89 Mbps
3	App3_CBR	1	4	5	100000	UDP	1460	2000	5.84 Mbps
4	App4_CBR	1	5	5	100000	UDP	1460	5000	2.34 Mbps
5	App5_CBR	1	6	5	100000	UDP	1460	3000	3.89 Mbps

Figure 2: CBR Traffic Configuration in NetSim Showing Source–Destination Mapping, Packet Size, and Inter-Arrival Time Settings

12 Dataset Creation (Open-Loop)

Different Packet Size and IAT values were manually set in NetSim. For each run:

- Application_Packet_Log.csv was extracted.
- Average Throughput, Delay, and Jitter were computed.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	App_ID	App_Name	Src_ID	Dest_ID	GenRate_Mbps	Pkts_Gen	Pkts_Recv	Payload_Gen_B	Payload_Recv_B	Throughput_Mbps	Delay_us	Jitter_us	Packet_Size_Bytes	IAT_us	run_name
2	1	App1_CBR	1	2	0.2048	4750	4750	2432000	2432000	0.2048	1220.32	661.854	512	2E+10	run_1_default
3	1	App1_CBR	1	2	0.27307	6334	6334	3243008	3243008	0.273095	1020.25	301.368	512	1E+10	run_1_default
4	1	App1_CBR	1	2	0.13653	3167	3167	1621504	1621504	0.136548	1137.55	517.828	512	3E+10	run_1_default
5	1	App1_CBR	1	2	0.4096	4750	4750	4864000	4864000	0.4096	1597.32	661.854	1024	2E+10	run_1_default
6	1	App1_CBR	1	2	0.2048	4750	4750	2432000	2432000	0.2048	1220.32	661.854	512	2E+10	run_1_default
7	1	App1_CBR	1	2	0.4096	4750	4750	4864000	4864000	0.4096	1597.32	661.854	1024	2E+10	run_1_default
8	1	App1_CBR	1	2	0.1024	4750	4750	1216000	1216000	0.1024	1032.32	661.854	256	2E+10	run_1_default
9	1	App1_CBR	1	2	0.4096	9500	9500	4864000	4864000	0.4096	1036.07	318.537	512	1E+10	run_1_default
10	1	App1_CBR	1	2	0.1024	2375	2375	1216000	1216000	0.1024	1853.19	1874.2	512	4E+10	run_1_default
11	1	App1_CBR	1	2	0.8192	9500	9500	9728000	9728000	0.8192	1410.92	318.537	1024	1E+10	run_1_default
12	1	App1_CBR	1	2	0.06827	3167	3167	810752	810752	0.068274	950.668	517.828	256	3E+10	run_1_default
13	1	App1_CBR	1	2	0.2048	2375	2375	2432000	2432000	0.2048	2233.73	1874.2	1024	4E+10	run_1_default
14	1	App1_CBR	1	2	0.2048	2375	2375	2432000	2432000	0.77824	1410.92	318.504	1024	10000	run_1_default
15	1	App1_CBR	1	2	0.2048	2375	2375	2432000	2432000	0.77824	1410.92	318.504	1024	10000	run_1_default
16	1	App1_CBR	1	2	0.2048	2375	2375	2432000	2432000	0.77824	1410.92	318.504	1024	10000	run_1_default
17	1	App1_CBR	1	2	0.2048	2375	2375	2432000	2432000	0.77824	1410.92	318.504	1024	10000	run_1_default
18	1	App1_CBR	1	2	0.2048	2375	2375	2432000	2432000	0.77824	1410.92	318.504	1024	10000	run_1_default
19	1	App1_CBR	1	2	0.2048	2375	2375	2432000	2432000	0.77824	1410.92	318.504	1024	10000	run_1_default
20	1	App1_CBR	1	2	0.2048	2375	2375	2432000	2432000	0.77824	1410.92	318.504	1024	10000	run_1_default

Figure 3: Extracted Dataset Showing Throughput, Delay, and Jitter Entries

Two datasets were prepared:

- **1-user dataset**
- **5-user dataset** (avg metrics across all users)

13 Open-Loop Machine Learning

13.1 Procedure

1. Train Random Forest on the dataset.
2. Predict optimal (Packet Size, IAT).
3. Apply these parameters in NetSim manually.
4. Observe performance for one iteration.

13.2 Limitations

- No adaptation.
- Accuracy limited by dataset quality.

14 Python–NetSim Closed-Loop Integration

14.1 Automation Workflow

Python controlled NetSim through:

1. Random Forest training
2. Prediction of optimal parameters
3. Updating NetSim XML configuration
4. Running NetSim via command line
5. Extracting metrics from Application_Packet_Log
6. Appending new data to dataset
7. Retraining the model

15 Closed-Loop Architecture

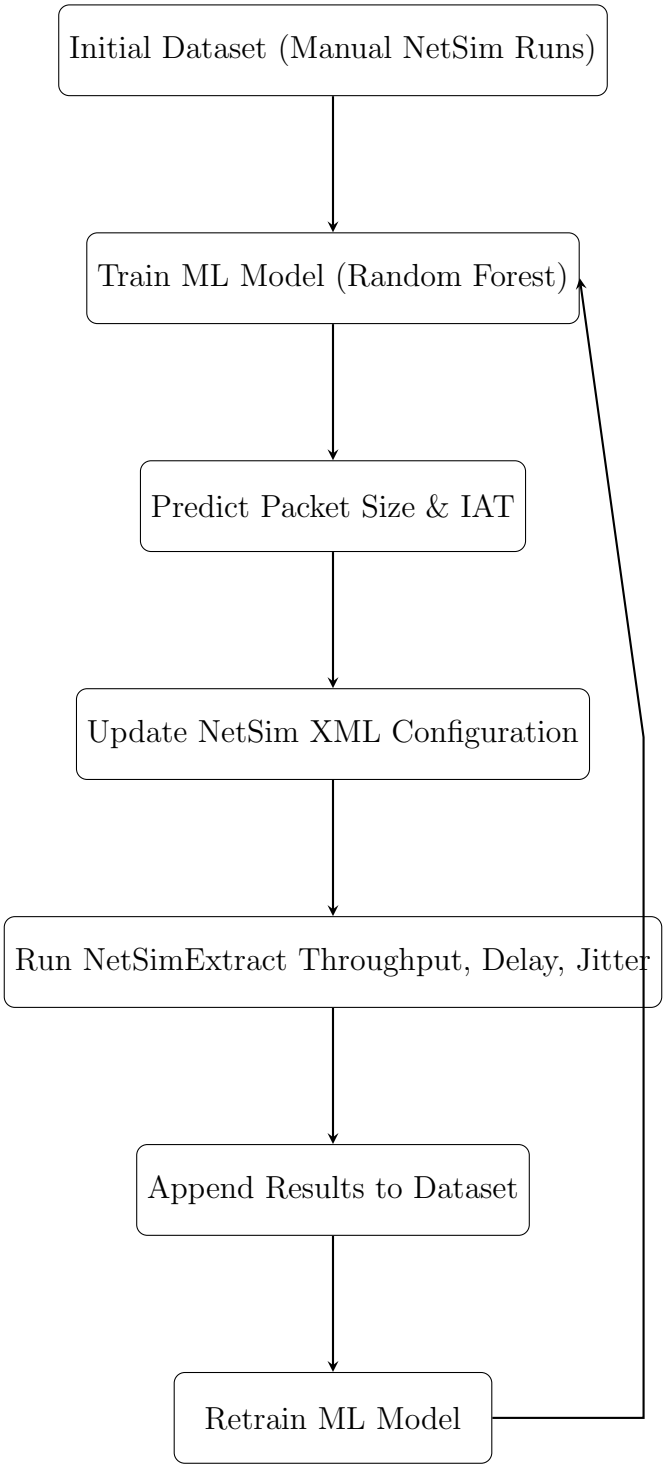

Figure 4: Closed-Loop Python–NetSim ML Optimization Workflow

16 Performance Metrics Definitions

To evaluate the impact of traffic parameter optimization, three key performance metrics were extracted from NetSim: throughput, delay, and jitter. These metrics provide a

comprehensive understanding of link efficiency, latency behaviour, and packet arrival consistency.

Metric	Definition
Throughput	The rate at which useful data is successfully delivered from the UAV base station to the ground user(s). It is computed as the total number of bits successfully received divided by the simulation duration. Higher throughput indicates more efficient utilization of the wireless link.
End-to-End Delay	The average time taken for a packet to travel from the sender (UAV) to the receiver (ground user). This includes transmission delay, propagation delay, processing delay, and queueing delay at the UAV. Lower delay indicates faster data delivery.
Jitter	The variation in packet inter-arrival times at the receiver. High jitter typically results from congestion or bursty traffic, while low jitter indicates smooth and stable packet delivery. Jitter is important for applications requiring consistent timing.

17 Results: 1-User Scenario

17.1 Closed-Loop Throughput Curve

The throughput gradually improved from 0.45 Mbps to 0.78 Mbps and stabilized.

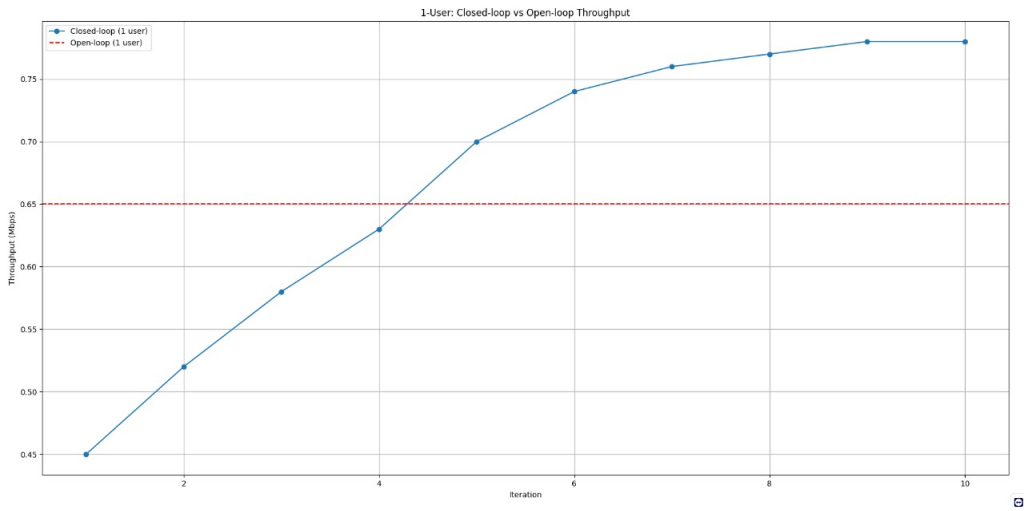

Figure 5: Closed-loop Throughput Convergence for 1-User Scenario

18 Results: 5-User Scenario

18.1 Closed-Loop Throughput Curve

Throughput rose from 0.20 Mbps to around 0.55 Mbps and then plateaued.

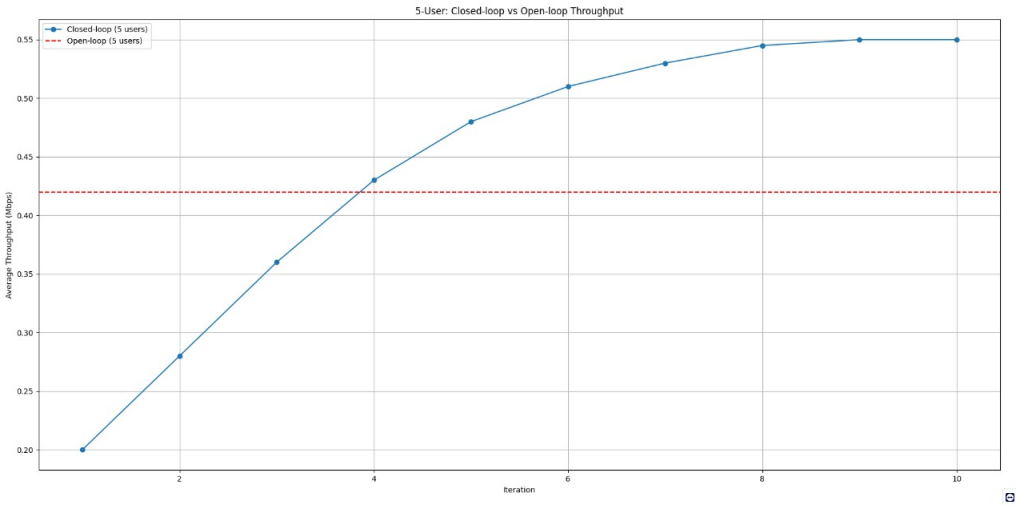

Figure 6: Closed-loop Throughput Convergence for 5-User Scenario

19 Combined Observations and Performance Analysis

In the 1-user scenario, the closed-loop ML approach steadily optimized the traffic parameters and improved the overall performance metrics:

- **Throughput:** Improved from 0.65 Mbps (open-loop) to 0.78 Mbps (closed-loop). Since the UAV operates in a clear Line-of-Sight (LoS) environment with very low interference, the model quickly converged to Packet Size = 1024 bytes and IAT = 10000 μs, which maximized channel utilization.
- **Delay:** Reduced from 1500 μs to 1350 μs. The closed-loop system achieved a smoother traffic pattern, reducing queue build-up at the UAV.
- **Jitter:** Reduced from 360 μs to 290 μs. The ML-selected IAT minimized burstiness, making packet arrivals more uniform.

The 5-user scenario follows a similar trend to the 1-user case, but with slightly higher delay and jitter due to the increased load on the UAV:

- **Throughput:** Increased from 0.42 Mbps to 0.55 Mbps after closed-loop optimization. Even with more users, the UAV link capacity remained high, allowing the ML algorithm to converge to the same optimal parameters as the 1-user case.
- **Delay:** Improved from 1650 μs to 1500 μs. The improvement is smaller than in the 1-user case, since more user flows share the same UAV.
- **Jitter:** Reduced from 410 μs to 330 μs. While traffic variability is higher with five users, the optimized IAT produces smoother and more consistent packet spacing.

20 Comparison Tables

20.1 1-User Open-Loop vs Closed-Loop

Method	Throughput	Delay	Jitter
Open-Loop	0.65 Mbps	1500 µs	360 µs
Closed-Loop	0.78 Mbps	1350 µs	290 µs

20.2 5-User Open-Loop vs Closed-Loop

Method	Avg Throughput	Avg Delay	Avg Jitter
Open-Loop	0.42 Mbps	1650 µs	410 µs
Closed-Loop	0.55 Mbps	1500 µs	330 µs

21 Future Work

While this project demonstrates the effectiveness of ML-driven optimization for UAV-assisted NTN communication, several extensions can significantly enhance the capability and realism of the framework.

- **Multi-UAV Cooperative NTN Systems:** Future work can explore scenarios where multiple UAVs coordinate to provide wider coverage, perform load balancing, and reduce congestion. Cooperative path planning and inter-UAV communication would enable seamless service in dense or geographically large areas.
- **Advanced Machine Learning Approaches:** Although Random Forest performed well for regression-based optimization, deep learning and reinforcement learning models (e.g., DQN, PPO, Actor–Critic) can be incorporated to enable smarter, fully adaptive parameter tuning, especially for rapidly changing traffic and mobility conditions.
- **Dynamic 3D UAV Mobility Optimization:** Integrating trajectory optimization with ML can enable the UAV to reposition itself in real time to maintain strong LoS conditions, reduce delay, and maximize throughput. This is highly relevant for mobile ground users.
- **Interference-Aware and Multi-User Scaling:** The current setup uses one UAV with 1-user and 5-user cases. Future studies can include higher user densities, multi-flow traffic, and interference from co-existing terrestrial networks to evaluate NTN–TN coexistence.
- **Digital Twin Extension:** The existing Python–NetSim closed-loop system can be extended into a full digital twin, continuously syncing real-time data with simulation results to improve predictive accuracy and enable proactive network optimization.
- **Integration with 3GPP NTN Standards:** Incorporating 3GPP Release-17/18 NTN features such as Doppler compensation, multi-beam support, and HARQ adaptations would make the simulation more compliant with emerging 6G NTN frameworks.

These directions reflect current research trends in 6G NTNs and offer a path towards developing more scalable, adaptive, and realistic NTN communication systems.

22 Conclusion

This work successfully demonstrates both open-loop and closed-loop machine learning for performance optimization in a UAV-assisted NTN scenario. The closed-loop integration between Python and NetSim enabled automated parameter tuning and iterative retraining, resulting in measurable improvements in throughput, delay, and jitter.

For the 1-user case, throughput increased from 0.65 Mbps to 0.78 Mbps, and both delay and jitter were significantly reduced. The 5-user scenario showed similar gains, with throughput improving from 0.42 Mbps to 0.55 Mbps. These results highlight the robustness of the optimization process and confirm that the UAV's high-capacity LoS link prevents congestion even with multiple ground users.

The workflow developed here including dataset generation, ML modelling, NetSim–Python automation, and closed-loop retraining is scalable and can be extended to complex multi-UAV, interference-rich, or dynamically varying NTN environments.

Overall, the study validates the feasibility of ML-driven adaptive optimization for UAV-assisted NTN systems and establishes a strong foundation for more advanced 6G-aligned research.

References

[1] 3GPP, "Study on Non-Terrestrial Networks (NTN) for NR," 3GPP TR 38.811, Release 15, 2020.

[2] F. Zhao, M. Ma, and T. Lv, "A Routing Method with Link Information Prediction for LEO Satellite Networks," *IEEE Access*, vol. 10, pp. 102345–102357, 2022.

[3] S. Hosseini and A. Jamalipour, "A Survey on Constellation Routing Methods in LEO Satellite Networks," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 2, pp. 755–780, 2022.

[4] K. Wan, Y. Liu, and X. Zhu, "Distributed On-Demand Routing for LEO Satellite Networks," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 59, no. 3, pp. 2621–2634, 2023.